

```

#-----
--#Esta função lê o ficheiro de configuração (config_S.txt) e retorna um dicionário com
os parâmetros da simulação.
def ler_config(S):
    filename = f"config_{S}.txt"
    config = {}
    with open(filename, 'r') as file:
        for line in file:
            line = line.strip()
            # Ignorar linhas vazias ou mal formatadas
            if not line or '=' not in line:
                continue
            key, value = line.split(' = ') # Divide chave e valor
            try:
                config[key] = eval(value) # Converte valores, como listas ou números
            except:
                config[key] = value # Mantém como string se não conseguir converter
    return config

#-----
--#Esta função cria um dicionário que representa uma pessoa, com um número de CC único,
nome, gênero, idade, salário e pensão.
CC = 1000000 # Número inicial de CC (global)

def nova_pessoa(idade=0):
    global CC
    pessoa = {
        "cc": CC,
        "nome": f"Pessoa_{CC}",
        "genero": "M" if CC % 2 == 0 else "F",
        "idade": idade,
        "salario": 1000 if 23 <= idade <= 67 else 0,
        "pensao": 500 if idade > 67 else 0
    }
    CC += 1
    return pessoa

#-----
--
#Esta função lê o ficheiro populacao_inicial_S.txt e cria a população com base nas
idades.
def ler_populacao_inicial(S):
    filename = f"populacao_inicial_{S}.txt"
    populacao = []
    with open(filename, 'r') as file:
        for line in file:
            partes = line.strip().split(', ') # Divide com base na vírgula e espaço
            idade = int(partes[3]) # A idade é o quarto valor
            populacao.append(nova_pessoa(idade))
    return populacao

```

```

#-----
--
#Esta função simula a passagem de um ano, atualiza as idades das pessoas, calcula a
mortalidade e adiciona novos nascimentos.

import random # Certifique-se de importar o módulo random

def simula_ano(populacao, ano_corrente, config):
    nova_populacao = []
    mortalidade = config["MORTALIDADE"]
    natalidade = config["NATALIDADE"]

    for pessoa in populacao:
        pessoa["idade"] += 1 # Aumenta a idade de cada pessoa
        idade = pessoa["idade"]

        # Atualiza salário e pensão com base na nova idade
        if 23 <= idade <= 67:
            pessoa["salario"] = 1000 # Trabalhadores têm salário
            pessoa["pensao"] = 0      # Trabalhadores não têm pensão
        elif idade > 67:
            pessoa["salario"] = 0      # Pensionistas não têm salário
            pessoa["pensao"] = 500     # Pensionistas recebem pensão
        else:
            pessoa["salario"] = 0      # Jovens não têm salário
            pessoa["pensao"] = 0       # Jovens não têm pensão

        # Verifica se a pessoa sobrevive com base na idade máxima
        if idade >= 100:
            continue # Remove a pessoa da população (não adiciona à nova_populacao)

        # Verifica se a pessoa sobrevive com base na probabilidade de mortalidade
        sobrevive = True
        for categoria, (faixa, prob) in mortalidade.items():
            if idade in faixa and random.random() < prob:
                sobrevive = False
                break

        if sobrevive:
            nova_populacao.append(pessoa) # Adiciona à nova população apenas os
sobreviventes

    # Adicionar novos nascimentos (com base na taxa de natalidade)
    for _ in range(len(populacao) // natalidade):
        nova_populacao.append(nova_pessoa(idade=0)) # Cria uma nova pessoa com idade 0

    return nova_populacao

```

```

#-----
--
def cobra_seg_social(ano, fundo_pensoes, populacao, config):
    descontos = config["DESCONTOS"]
    total_contribuicoes = 0
    total_pensoes = 0

    for pessoa in populacao:
        if 23 <= pessoa["idade"] <= 67: # Adultos contribuem
            total_contribuicoes += pessoa["salario"] * descontos
        elif pessoa["idade"] > 67: # Pensionistas recebem
            total_pensoes += pessoa["pensao"]

    fundo_pensoes += total_contribuicoes - total_pensoes
    return fundo_pensoes

#-----
--
def simular(S):
    # Ler configurações e população inicial
    config = ler_config(S)
    populacao = ler_populacao_inicial(S)
    fundo_pensoes = config["FUNDO_PENSOES_INICIAL"]
    epocas = config["EPOCAS"]
    ano = config["ANO_INICIAL"]

    print(f"A simulação começou no ano {ano}, com população total de {len(populacao)}, e
o fundo de pensões a valer {fundo_pensoes}")

    for i in range(epocas):
        ano += 1
        populacao = simula_ano(populacao, ano, config)
        fundo_pensoes = cobra_seg_social(ano, fundo_pensoes, populacao, config)

        # Agora imprime o fundo e a população a cada ano
        print(f"No ano {ano}, a população foi {len(populacao)} e o fundo de pensões vale
{fundo_pensoes}")

        if fundo_pensoes < 0:
            print(f"No ano {ano}, o fundo de pensões ficou negativo, com valor
{fundo_pensoes}")

    print(f"A simulação terminou no ano {ano}, com população total de {len(populacao)}
pessoas e o fundo de pensões vale {fundo_pensoes}")

    # Salvar a população final no formato correto
    with open(f"populacao_final_{S}.txt", 'w') as file:
        for pessoa in populacao:

```

```
file.write(f"{pessoa['cc']}, {pessoa['nome']}, {pessoa['genero']},  
{pessoa['idade']}, {pessoa['salario']}, {pessoa['pensao']}\n")
```

```
#-----  
--
```

```
if __name__ == "__main__":  
    S = 11 # Código da simulação  
    simular(S)
```

```
#-----  
-----  
----
```

A simulação começou no ano 2011, com população total de 10000, e o fundo de pensões a valer 10000000

No ano 2012,	a população foi	9822	e o fundo de pensões vale	9430940.0
No ano 2013,	a população foi	9723	e o fundo de pensões vale	8881720.0
No ano 2014,	a população foi	9611	e o fundo de pensões vale	8366590.0
No ano 2015,	a população foi	9531	e o fundo de pensões vale	7881700.0
No ano 2016,	a população foi	9450	e o fundo de pensões vale	7413230.0
No ano 2017,	a população foi	9362	e o fundo de pensões vale	6975200.0
No ano 2018,	a população foi	9290	e o fundo de pensões vale	6561300.0
No ano 2019,	a população foi	9216	e o fundo de pensões vale	6183860.0
No ano 2020,	a população foi	9165	e o fundo de pensões vale	5814780.0
No ano 2021,	a população foi	9118	e o fundo de pensões vale	5462170.0
No ano 2022,	a população foi	9061	e o fundo de pensões vale	5141550.0
No ano 2023,	a população foi	8985	e o fundo de pensões vale	4847170.0
No ano 2024,	a população foi	8935	e o fundo de pensões vale	4577250.0
No ano 2025,	a população foi	8885	e o fundo de pensões vale	4318340.0
No ano 2026,	a população foi	8836	e o fundo de pensões vale	4067170.0
No ano 2027,	a população foi	8801	e o fundo de pensões vale	3823240.0
No ano 2028,	a população foi	8760	e o fundo de pensões vale	3585430.0
No ano 2029,	a população foi	8733	e o fundo de pensões vale	3350240.0
No ano 2030,	a população foi	8689	e o fundo de pensões vale	3114290.0
No ano 2031,	a população foi	8660	e o fundo de pensões vale	2876460.0
No ano 2032,	a população foi	8622	e o fundo de pensões vale	2647520.0
No ano 2033,	a população foi	8595	e o fundo de pensões vale	2427240.0
No ano 2034,	a população foi	8556	e o fundo de pensões vale	2221190.0
No ano 2035,	a população foi	8523	e o fundo de pensões vale	2026060.0
No ano 2036,	a população foi	8489	e o fundo de pensões vale	1828750.0
No ano 2037,	a população foi	8468	e o fundo de pensões vale	1627410.0
No ano 2038,	a população foi	8452	e o fundo de pensões vale	1425380.0
No ano 2039,	a população foi	8433	e o fundo de pensões vale	1235920.0
No ano 2040,	a população foi	8398	e o fundo de pensões vale	1056080.0
No ano 2041,	a população foi	8366	e o fundo de pensões vale	881210.0

A simulação terminou no ano 2041, com população total de 8366 pessoas e o fundo de pensões vale 881210.0

ficheiro final

1000000, Pessoa_1000000, M, 75, 0, 500
1000001, Pessoa_1000001, F, 38, 1000, 0
1000002, Pessoa_1000002, M, 85, 0, 500
1000004, Pessoa_1000004, M, 88, 0, 500
1000006, Pessoa_1000006, M, 73, 0, 500
1000007, Pessoa_1000007, F, 62, 1000, 0
1000008, Pessoa_1000008, M, 95, 0, 500
1000009, Pessoa_1000009, F, 79, 0, 500
1000012, Pessoa_1000012, M, 35, 1000, 0
1000013, Pessoa_1000013, F, 83, 0, 500
1000015, Pessoa_1000015, F, 33, 1000, 0
1000016, Pessoa_1000016, M, 43, 1000, 0

.....

1009296, Pessoa_1009296, M, 46, 1000, 0
1009297, Pessoa_1009297, F, 50, 1000, 0
1009298, Pessoa_1009298, M, 78, 0, 500
1009299, Pessoa_1009299, F, 88, 0, 500
1009300, Pessoa_1009300, M, 91, 0, 500
1009301, Pessoa_1009301, F, 73, 0, 500
1009303, Pessoa_1009303, F, 34, 1000, 0
1009306, Pessoa_1009306, M, 37, 1000, 0
1009307, Pessoa_1009307, F, 55, 1000, 0
1009308, Pessoa_1009308, M, 78, 0, 500

.....

1010670, Pessoa_1010670, M, 23, 1000, 0
1010671, Pessoa_1010671, F, 23, 1000, 0
1010672, Pessoa_1010672, M, 23, 1000, 0
1010673, Pessoa_1010673, F, 22, 0, 0
1010674, Pessoa_1010674, M, 22, 0, 0
1010675, Pessoa_1010675, F, 22, 0, 0
1010676, Pessoa_1010676, M, 22, 0, 0
1010677, Pessoa_1010677, F, 22, 0, 0
1010678, Pessoa_1010678, M, 22, 0, 0
1010679, Pessoa_1010679, F, 22, 0, 0
1010680, Pessoa_1010680, M, 22, 0, 0
1010681, Pessoa_1010681, F, 22, 0, 0
1010682, Pessoa_1010682, M, 22, 0, 0
1010683, Pessoa_1010683, F, 22, 0, 0
1010684, Pessoa_1010684, M, 22, 0, 0